Criticisms of Scott–Strachey (Denotational) Semantics

ScottLean4 · for Dana Scott

Denotational semantics gave the field its first rigorous, compositional notion of program meaning, and its account of recursion via least fixed points. The criticisms below are the substantive limitations that have been raised against it; much of programming-language semantics since can be read as a response to them.

1. Full abstraction — the central technical critique

The standard continuous-function (Scott-domain) model of PCF is **not fully abstract** (Plotkin and Milner, 1977): it contains elements — famously **parallel-or** — that are *not definable* by any program, so the model distinguishes terms that are operationally indistinguishable. Denotational equality is strictly *finer* than observational equivalence; the model has "junk." Restoring the exact match required either extending the language (add parallel-or) or far more intricate models: **sequential algorithms** (Berry–Curien) and, decades later, **game semantics** (Abramsky–Jagadeesan–Malacaria; Hyland–Ong). That it took roughly twenty years to model a tiny language *exactly* is the most-cited limitation.

2. Continuity captures too much — sequentiality escapes it

Continuity is weaker than sequential computability. Continuous functions include inherently parallel ones, so the order-theoretic notion of a computable function overshoots what a sequential machine can do. Capturing sequentiality needed extra structure — Kahn–Plotkin sequential functions, Berry's **stable functions and dI-domains**, coherence spaces — none of it forced by the original domain axioms.

3. It is extensional — the intensional life of a program is lost

A program denotes an input–output *function*, so two programs computing the same function have the **same denotation even if one is exponentially slower**. Denotational semantics therefore says nothing directly about **cost, evaluation order, or complexity**: the entire intensional dimension is abstracted away. To reason about *how* a program computes, one is pushed back to operational accounts.

4. Effects, nondeterminism, and concurrency are heavy or ad hoc

Control needed **continuations** (Strachey–Wadsworth); state needed threading a store; nondeterminism needed **powerdomains** (Plotkin, Smyth) — and no powerdomain construction validates all the equivalences one wants. **True concurrency** is served poorly by domain theory, which drove

the separate development of process calculi and event structures. Each effect originally demanded bespoke domain surgery, later reorganized more cleanly by **Moggi's monads** (1989) — itself an implicit criticism that the original treatment of effects was unstructured.

5. The mathematics is heavy — and operational semantics is often "more real"

Solving recursive domain equations such as $D \cong [D \to D]$ (inverse limits, embedding–projection pairs, algebraically compact categories) is sophisticated, and the semantics of a real language becomes a large, unwieldy object. This accessibility cost contributed to the rise of **structural operational semantics** (Plotkin, 1981) as simpler, more direct, and — for many — *primary*, with denotational semantics relegated to a harder-to-build complement that must be tied back by a **computational-adequacy** proof. Later tools (logical relations, bisimulation, step-indexing) reinforced the operational-first stance.

6. Deeper reservations

- **Compositionality**, its great virtue, is also a straitjacket: some features (dynamic scope, reflection, unrestricted concurrency) resist a natural compositional denotation.
- A **philosophical** worry: a Scott-domain element is *one mathematical model*, laden with modeling choices (the treatment of $\perp$, the "extra" limit points), not a canonical account of "meaning" — so the claim that it *is* the meaning of a program is contestable.

The through-line

Denotational semantics was an enormous success, but (i) the naïve models do not exactly match observation (full abstraction), (ii) continuity $\neq$ sequentiality, (iii) it is blind to intensional cost, and (iv) effects and concurrency strain the machinery. Game semantics, monads, and operational methods are, in large part, the field's answers to precisely these points.